Testing and Accessibility

Comprehensive testing and accessibility ensure your chatbot application works reliably for all users across all devices and assistive technologies. This section creates detailed specifications for testing strategies, accessibility compliance, and quality assurance processes.

Exercise 4.1: Comprehensive Testing Strategy

Create detailed testing specifications covering all aspects of frontend functionality:

~~~
Design comprehensive testing strategy for our chatbot application:

**Use the NPM Registry MCP server to research current testing tools and frameworks, then create:**

**Testing Pyramid Implementation**:
```typescript
interface TestingPyramid {
  // Unit Tests (70% of tests)
  unitTests: {
    framework: "Jest with React Testing Library";
    coverage: "Minimum 80% code coverage";
    scope: [
      "Individual component behavior",
      "Custom hooks functionality",
      "Utility functions",
      "State management logic"
    ];
    patterns: [
      "Test component props and rendering",
      "Test user interactions and events",
      "Test component state changes",
      "Test error boundaries and fallbacks"
    ];
  };

  // Integration Tests (20% of tests)
  integrationTests: {
    framework: "React Testing Library with MSW";
    scope: [
      "Component integration with APIs",
      "Multi-component workflows",
      "Context provider integration",
      "Route-based component testing"
    ];
    patterns: [
      "Test complete user flows",
      "Test API integration with mocked responses",
      "Test component communication",
      "Test state synchronization"
    ];
  };

  // End-to-End Tests (10% of tests)
  e2eTests: {
    framework: "Playwright for cross-browser testing";
    scope: [
      "Critical user journeys",
      "Cross-browser compatibility",
      "Performance validation",
      "Accessibility validation"
    ];
    patterns: [
      "Test complete application workflows",
      "Test real API integration",
      "Test responsive design",
      "Test accessibility features"
    ];
  };
}
```

**Component Testing Specifications**:
```typescript
// Chat Component Testing
interface ChatComponentTests {
  messageComponent: {
    rendering: [
      "Renders user messages with correct styling",
      "Renders assistant messages with correct styling",
      "Renders system messages appropriately",
      "Handles long messages with proper wrapping"
    ];

    interactions: [
      "Handles message selection and copying",
      "Handles message editing functionality",
      "Handles message deletion with confirmation",
      "Handles retry functionality for failed messages"
    ];

    states: [
      "Shows loading state during message sending",
      "Shows error state for failed messages",
      "Shows typing indicator appropriately",
      "Handles empty conversation state"
    ];
  };

  inputComponent: {
    validation: [
      "Prevents sending empty messages",
      "Enforces character limits",
      "Validates message content",
      "Shows appropriate error messages"
    ];

    interactions: [
      "Handles Enter key to send message",
      "Handles Shift+Enter for new lines",
      "Handles paste operations correctly",
      "Auto-expands textarea appropriately"
    ];

    accessibility: [
      "Proper ARIA labels and descriptions",
      "Keyboard navigation support",
      "Screen reader announcements",
      "Focus management"
    ];
  };
}
```

**API Integration Testing**:
```typescript
interface APIIntegrationTests {
  // Mock Service Worker Configuration
  mswConfiguration: {
    handlers: [
      "Authentication endpoints (login, logout, refresh)",
      "Chat endpoints (send message, get history)",
      "User management endpoints",
      "Error scenarios and edge cases"
    ];

    scenarios: [
      "Successful API responses",
      "Network errors and timeouts",
      "Authentication failures",
      "Rate limiting responses",
      "Server errors (500, 502, 503)"
    ];
  };

  // Integration Test Patterns
  integrationPatterns: {
    authFlow: "Test complete authentication workflow";
    chatFlow: "Test message sending and receiving";
    errorHandling: "Test error scenarios and recovery";
    offlineMode: "Test offline functionality and sync";
  };
}
```

Research current React testing best practices and tools using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive testing pyramid with appropriate test distribution
- Detailed component testing specifications for all UI elements
- API integration testing with comprehensive mock scenarios
- Testing patterns for user workflows and error handling
- Performance and accessibility testing integration#

Exercise 4.2: Accessibility Testing and Validation

Create comprehensive accessibility testing specifications ensuring WCAG 2.1 AA compliance:

~~~
Design detailed accessibility testing framework for our chatbot application:

**Automated Accessibility Testing**:
```typescript
interface AutomatedA11yTesting {
  // Jest-axe Integration
  jestAxeIntegration: {
    setup: "Configure jest-axe for component testing";
    rules: "Custom axe rules for chatbot-specific patterns";
    coverage: "Test all components for accessibility violations";

    testPatterns: [
      "Test component renders without a11y violations",
      "Test interactive states (focus, hover, active)",
      "Test dynamic content updates",
      "Test form validation and error states"
    ];
  };

  // Lighthouse Accessibility Audits
  lighthouseA11y: {
    automation: "Automated Lighthouse accessibility audits in CI";
    thresholds: "Minimum 95% accessibility score";
    reporting: "Detailed accessibility audit reports";

    checks: [
      "Color contrast ratios",
      "Keyboard navigation",
      "Screen reader compatibility",
      "ARIA implementation",
      "Semantic HTML structure"
    ];
  };

  // Pa11y Integration
  pa11yTesting: {
    configuration: "Pa11y for command-line accessibility testing";
    standards: "WCAG 2.1 AA compliance testing";
    integration: "CI/CD pipeline integration";

    scenarios: [
      "Test all major pages and routes",
      "Test interactive components",
      "Test form submissions and validation",
      "Test dynamic content updates"
    ];
  };
}
```

**Manual Accessibility Testing**:
```typescript
interface ManualA11yTesting {
  // Keyboard Navigation Testing
  keyboardTesting: {
    navigation: [
      "Tab through all interactive elements in logical order",
      "Test skip links functionality",
      "Test keyboard shortcuts and access keys",
      "Test focus management in modals and overlays"
    ];

    chatInterface: [
      "Navigate message history with arrow keys",
      "Access message actions via keyboard",
      "Navigate to and from chat input efficiently",
      "Test keyboard shortcuts for common actions"
    ];

    forms: [
      "Navigate form fields in logical order",
      "Test form validation with keyboard only",
      "Test error message accessibility",
      "Test form submission via keyboard"
    ];
  };

  // Screen Reader Testing
  screenReaderTesting: {
    tools: ["NVDA (Windows)", "JAWS (Windows)", "VoiceOver (macOS)", "Orca (Linux)"];

    scenarios: [
      "Navigate entire application with screen reader",
      "Test chat message announcements",
      "Test form field labels and descriptions",
      "Test error message announcements",
      "Test dynamic content updates"
    ];

    validation: [
      "All content is accessible via screen reader",
      "Proper heading structure and landmarks",
      "Meaningful link and button descriptions",
      "Appropriate use of ARIA labels and descriptions"
    ];
  };

  // Visual Accessibility Testing
  visualTesting: {
    colorBlindness: [
      "Test with deuteranopia simulation",
      "Test with protanopia simulation",
      "Test with tritanopia simulation",
      "Ensure information isn't conveyed by color alone"
    ];

    lowVision: [
      "Test at 200% zoom level",
      "Test with high contrast mode",
      "Test with custom color schemes",
      "Test with reduced motion preferences"
    ];
  };
}
```

**Accessibility Implementation Validation**:
```typescript
interface A11yImplementationValidation {
  // ARIA Implementation
  ariaValidation: {
    roles: [
      "Validate proper ARIA roles usage",
      "Test custom component roles",
      "Validate landmark roles",
      "Test interactive element roles"
    ];

    properties: [
      "Test aria-label and aria-labelledby",
      "Test aria-describedby for descriptions",
      "Test aria-expanded for collapsible content",
      "Test aria-live for dynamic updates"
    ];

    states: [
      "Test aria-disabled for disabled elements",
      "Test aria-selected for selectable items",
      "Test aria-checked for checkboxes",
      "Test aria-pressed for toggle buttons"
    ];
  };

  // Semantic HTML Validation
  semanticValidation: {
    structure: [
      "Proper heading hierarchy (h1-h6)",
      "Semantic landmarks (main, nav, aside)",
      "List structures for grouped content",
      "Table structures for tabular data"
    ];

    forms: [
      "Proper label associations",
      "Fieldset and legend for grouped inputs",
      "Required field indicators",
      "Error message associations"
    ];
  };
}
```

Research current accessibility testing tools and methodologies using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive automated accessibility testing with multiple tools
- Manual testing procedures for keyboard navigation and screen readers
- Visual accessibility testing for color blindness and low vision
- ARIA implementation validation and semantic HTML verification
- Continuous accessibility monitoring and compliance validation

Exercise 4.3: Cross-Browser and Device Testing

Create comprehensive cross-browser and device testing specifications:

~~~
Design detailed cross-browser and device testing strategy:

**Browser Compatibility Matrix**:
```typescript
interface BrowserCompatibility {
  // Desktop Browser Support
  desktopBrowsers: {
    chrome: {
      versions: "Latest 2 versions";
      features: "Full feature support";
      testing: "Primary development browser";
    };

    firefox: {
      versions: "Latest 2 versions";
      features: "Full feature support";
      testing: "Secondary testing browser";
    };

    safari: {
      versions: "Latest 2 versions";
      features: "Full feature support with WebKit considerations";
      testing: "macOS and iOS testing";
    };

    edge: {
      versions: "Latest 2 versions";
      features: "Full feature support";
      testing: "Windows compatibility";
    };
  };

  // Mobile Browser Support
  mobileBrowsers: {
    iosSafari: {
      versions: "iOS 14+ (latest 3 versions)";
      features: "Touch-optimized interface";
      testing: "iPhone and iPad testing";
    };

    androidChrome: {
      versions: "Android 8+ (latest 3 versions)";
      features: "Touch-optimized interface";
      testing: "Various Android devices";
    };

    samsungInternet: {
      versions: "Latest 2 versions";
      features: "Samsung-specific optimizations";
      testing: "Samsung device testing";
    };
  };
}
```

**Device Testing Strategy**:
```typescript
interface DeviceTestingStrategy {
  // Physical Device Testing
  physicalDevices: {
    smartphones: [
      "iPhone 13/14 (iOS Safari)",
      "Samsung Galaxy S22/S23 (Chrome)",
      "Google Pixel 6/7 (Chrome)",
      "OnePlus 9/10 (Chrome)"
    ];

    tablets: [
      "iPad Air/Pro (iOS Safari)",
      "Samsung Galaxy Tab (Chrome)",
      "Surface Pro (Edge)"
    ];

    desktops: [
      "Windows 10/11 (Chrome, Edge, Firefox)",
      "macOS Monterey/Ventura (Safari, Chrome)",
      "Ubuntu Linux (Firefox, Chrome)"
    ];
  };

  // Browser Testing Tools
  browserTestingTools: {
    playwright: {
      purpose: "Automated cross-browser testing";
      browsers: "Chromium, Firefox, WebKit";
      features: "Screenshots, network interception, mobile emulation";
    };

    browserStack: {
      purpose: "Real device and browser testing";
      coverage: "2000+ browser/device combinations";
      features: "Live testing, automated testing, debugging";
    };

    lambdaTest: {
      purpose: "Cross-browser compatibility testing";
      features: "Live testing, screenshot testing, responsive testing";
    };
  };

  // Responsive Design Testing
  responsiveDesignTesting: {
    viewports: [
      "320px (iPhone SE)",
      "375px (iPhone 12/13)",
      "414px (iPhone 12/13 Pro Max)",
      "768px (iPad)",
      "1024px (iPad Pro)",
      "1280px (Desktop)",
      "1920px (Large Desktop)"
    ];

    orientations: ["Portrait", "Landscape"];

    features: [
      "Layout adaptation",
      "Touch target sizing",
      "Text readability",
      "Image scaling",
      "Navigation usability"
    ];
  };
}
```

**Performance Testing Across Devices**:
```typescript
interface CrossDevicePerformance {
  // Device Performance Categories
  deviceCategories: {
    highEnd: {
      examples: "iPhone 13+, Samsung Galaxy S22+, Desktop";
      expectations: "Full performance, all features enabled";
      testing: "Baseline performance testing";
    };

    midRange: {
      examples: "iPhone 11, Samsung Galaxy A52, Older laptops";
      expectations: "Good performance with minor optimizations";
      testing: "Performance validation with throttling";
    };

    lowEnd: {
      examples: "iPhone 8, Budget Android, Older devices";
      expectations: "Acceptable performance with feature degradation";
      testing: "Heavy throttling and memory constraints";
    };
  };

  // Network Condition Testing
  networkTesting: {
    conditions: [
      "Fast 3G (1.6 Mbps down, 750 Kbps up)",
      "Slow 3G (400 Kbps down, 400 Kbps up)",
      "2G (280 Kbps down, 256 Kbps up)",
      "Offline (Service worker testing)"
    ];

    scenarios: [
      "Initial app load on slow networks",
      "Message sending with poor connectivity",
      "Offline mode and sync functionality",
      "Progressive loading and caching"
    ];
  };
}
```

Research current cross-browser testing tools and device testing strategies using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive browser compatibility matrix with version support
- Device testing strategy covering smartphones, tablets, and desktops
- Automated cross-browser testing with Playwright and cloud services
- Responsive design testing across multiple viewports and orientations
- Performance testing across different device categories and network conditions

Exercise 4.4: Visual Regression and UI Testing

Create comprehensive visual regression testing specifications:

~~~
Design detailed visual regression testing framework:

**Visual Testing Strategy**:
```typescript
interface VisualTestingStrategy {
  // Chromatic Integration
  chromaticTesting: {
    purpose: "Visual regression testing for components";
    coverage: "All components and their variants";
    automation: "Automated visual testing in CI/CD";

    configuration: {
      projectToken: "Chromatic project token";
      buildScriptName: "chromatic";
      exitZeroOnChanges: true;
      autoAcceptChanges: "main";
    };

    scenarios: [
      "Component variants and states",
      "Responsive breakpoints",
      "Theme variations (light/dark)",
      "Accessibility states (focus, hover)"
    ];
  };

  // Percy Integration
  percyTesting: {
    purpose: "Visual testing for full pages";
    coverage: "All major pages and user flows";
    browsers: "Chrome, Firefox, Safari";

    configuration: {
      widths: [375, 768, 1280];
      minHeight: 1024;
      percyCSS: "Hide dynamic content for consistent screenshots";
    };

    scenarios: [
      "Authentication pages",
      "Chat interface states",
      "Settings and profile pages",
      "Error and loading states"
    ];
  };

  // Playwright Visual Testing
  playwrightVisual: {
    purpose: "Custom visual regression testing";
    implementation: "expect(page).toHaveScreenshot()";
    configuration: "Threshold and animation handling";

    testPatterns: [
      "Full page screenshots",
      "Component-specific screenshots",
      "Mobile vs desktop comparisons",
      "Before/after interaction screenshots"
    ];
  };
}
```

**Screenshot Testing Best Practices**:
```typescript
interface ScreenshotTesting {
  // Consistent Screenshot Conditions
  consistency: {
    environment: "Consistent browser and OS versions";
    timing: "Wait for animations and loading to complete";
    data: "Use consistent test data and timestamps";
    fonts: "Use web fonts or consistent system fonts";
  };

  // Dynamic Content Handling
  dynamicContent: {
    timestamps: "Mock or hide timestamps";
    userContent: "Use consistent test user data";
    animations: "Disable or wait for animations";
    randomContent: "Mock random elements";
  };

  // Threshold Configuration
  thresholdConfiguration: {
    pixelThreshold: "0.2% pixel difference threshold";
    colorThreshold: "Small color variation tolerance";
    antialiasing: "Handle antialiasing differences";
    fontRendering: "Account for font rendering differences";
  };
}
```

**Component Story Testing**:
```typescript
interface ComponentStoryTesting {
  // Storybook Integration
  storybookIntegration: {
    purpose: "Component documentation and testing";
    stories: "Comprehensive stories for all components";
    addons: [
      "@storybook/addon-a11y",
      "@storybook/addon-viewport",
      "@storybook/addon-controls",
      "chromatic"
    ];
  };

  // Story Coverage
  storyCoverage: {
    variants: "All component variants and props";
    states: "All interactive states";
    data: "Various data scenarios";
    responsive: "Different viewport sizes";
  };

  // Interaction Testing
  interactionTesting: {
    framework: "@storybook/testing-library";
    scenarios: [
      "User interactions within stories",
      "Form submissions and validation",
      "Modal and dropdown interactions",
      "Keyboard navigation testing"
    ];
  };
}
```

Research current visual regression testing tools and best practices using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive visual regression testing with multiple tools
- Consistent screenshot testing practices and configuration
- Component story testing with Storybook integration
- Automated visual testing in CI/CD pipeline
- Dynamic content handling and threshold configuration

Exercise 4.5: Quality Assurance and Test Automation

Create comprehensive QA processes and test automation framework:

~~~
Design detailed quality assurance and test automation specifications:

**Continuous Integration Testing**:
```typescript
interface CITestingPipeline {
  // Pre-commit Hooks
  preCommitHooks: {
    linting: "ESLint and Prettier formatting";
    typeChecking: "TypeScript compilation check";
    unitTests: "Fast unit tests execution";
    accessibility: "Basic accessibility checks";
  };

  // Pull Request Checks
  pullRequestChecks: {
    fullTestSuite: "All unit and integration tests";
    e2eTests: "Critical path end-to-end tests";
    visualRegression: "Chromatic visual testing";
    performanceTests: "Lighthouse performance audits";
    accessibilityTests: "Comprehensive a11y validation";

    qualityGates: {
      testCoverage: "> 80%";
      performanceScore: "> 90";
      accessibilityScore: "> 95";
      bundleSize: "Within performance budget";
    };
  };

  // Deployment Testing
  deploymentTesting: {
    stagingTests: "Full test suite on staging environment";
    smokeTests: "Critical functionality validation";
    performanceTests: "Real environment performance testing";
    accessibilityTests: "Production accessibility validation";
  };
}
```

**Test Data Management**:
```typescript
interface TestDataManagement {
  // Test Data Strategy
  testDataStrategy: {
    fixtures: "Static test data for consistent testing";
    factories: "Dynamic test data generation";
    mocking: "API response mocking with MSW";
    cleanup: "Automated test data cleanup";
  };

  // Data Generation
  dataGeneration: {
    users: "Generate test users with various profiles";
    conversations: "Generate conversation history";
    messages: "Generate messages with various content";
    errors: "Generate error scenarios and edge cases";
  };

  // Environment Management
  environmentManagement: {
    isolation: "Isolated test environments";
    seeding: "Consistent data seeding";
    reset: "Environment reset between test runs";
    monitoring: "Test environment health monitoring";
  };
}
```

**Quality Metrics and Reporting**:
```typescript
interface QualityMetrics {
  // Test Metrics
  testMetrics: {
    coverage: "Code coverage percentage and trends";
    execution: "Test execution time and reliability";
    flakiness: "Flaky test identification and resolution";
    maintenance: "Test maintenance effort and cost";
  };

  // Quality Dashboards
  qualityDashboards: {
    testResults: "Real-time test execution results";
    coverage: "Code coverage trends and gaps";
    performance: "Performance metrics over time";
    accessibility: "Accessibility compliance tracking";
  };

  // Reporting and Alerts
  reportingAlerts: {
    testFailures: "Immediate alerts for test failures";
    coverageDrops: "Alerts for coverage decreases";
    performanceRegression: "Performance degradation alerts";
    accessibilityIssues: "New accessibility violations";
  };
}
```

**Test Maintenance and Optimization**:
```typescript
interface TestMaintenance {
  // Test Optimization
  testOptimization: {
    parallelization: "Parallel test execution for speed";
    sharding: "Test sharding across multiple workers";
    caching: "Test result and dependency caching";
    selection: "Smart test selection based on changes";
  };

  // Maintenance Processes
  maintenanceProcesses: {
    regularReview: "Monthly test suite review and cleanup";
    flakeReduction: "Systematic flaky test resolution";
    performanceOptimization: "Test execution performance tuning";
    documentation: "Test documentation and knowledge sharing";
  };

  // Continuous Improvement
  continuousImprovement: {
    metrics: "Track test effectiveness and efficiency";
    feedback: "Developer feedback on testing experience";
    tooling: "Regular evaluation of testing tools";
    training: "Team training on testing best practices";
  };
}
```

Set up Agent Hooks to automatically run tests when code is committed, update test documentation, and generate quality reports.

Research current test automation and QA best practices using the NPM Registry MCP server.
~~~

Expected Outcome

- Comprehensive CI/CD testing pipeline with quality gates
- Test data management strategy with fixtures and factories
- Quality metrics and reporting dashboards
- Test maintenance and optimization processes
- Continuous improvement framework for testing practices

Key Deliverables

After completing the Testing and Accessibility exercises, you should have:

1. **Comprehensive Testing Strategy**: Unit, integration, and e2e testing with proper coverage
2. **Accessibility Compliance**: WCAG 2.1 AA compliance with automated and manual testing
3. **Cross-Browser Compatibility**: Testing across all major browsers and devices
4. **Visual Regression Testing**: Automated visual testing with multiple tools
5. **Quality Assurance Framework**: CI/CD integration with quality gates and metrics

Integration with Advanced Features

Your testing and accessibility framework leverages:

- **NPM Registry MCP Server**: For researching testing tools and accessibility libraries
- **Web Search MCP Server**: For current testing and accessibility best practices
- **Agent Hooks**: For automated test execution and quality validation
- **Agent Steering**: For applying testing standards and accessibility requirements

Frontend Phase Completion

With the Frontend phase complete, you have established:

- **Design System**: Comprehensive visual design and component patterns
- **Component Architecture**: Scalable React architecture with TypeScript
- **Performance Optimization**: Bundle optimization, caching, and PWA features
- **Testing and Accessibility**: Comprehensive testing and WCAG 2.1 AA compliance

Next Steps

Your frontend specifications provide a complete blueprint for building an accessible, performant, and well-tested user interface. The next phases will focus on:

1. **Backend Specifications**: API design, business logic, and data management
2. **Security Specifications**: Application security, authentication, and compliance
3. **Implementation and Testing**: Bringing all specifications together

Each phase will integrate with your frontend specifications to create a cohesive, full-stack chatbot application that delivers exceptional user experiences while maintaining security and reliability.#

Exercise 4.6: Frontend Implementation

Implement the complete frontend based on all specifications:

~~~
Based on all frontend specifications, implement the React chatbot application:

**Step 1: Design System Implementation**
Create the complete design system based on specifications:

```typescript
// src/design-system/tokens.ts
export const designTokens = {
  colors: {
    primary: {
      50: '#f0f9ff',
      500: '#0ea5e9',
      900: '#0c4a6e'
    },
    // Implement all color tokens as specified
  },
  typography: {
    fontFamilies: {
      primary: 'Inter, -apple-system, BlinkMacSystemFont, sans-serif'
    },
    // Implement all typography tokens as specified
  },
  spacing: {
    // Implement all spacing tokens as specified
  }
};

// src/components/ui/Button.tsx
interface ButtonProps {
  variant: 'primary' | 'secondary' | 'outline';
  size: 'sm' | 'md' | 'lg';
  // Implement all props as specified
}

export const Button: React.FC<ButtonProps> = ({ variant, size, ...props }) => {
  // Implement component exactly as specified in design system
};
```

**Step 2: Component Architecture Implementation**
Build the complete component hierarchy as specified:

```typescript
// src/components/Chat/ChatInterface.tsx
export const ChatInterface: React.FC = () => {
  // Implement using specifications:
  // - Message list with virtual scrolling
  // - Input component with validation
  // - Real-time updates and typing indicators
  // - Error handling and retry logic
};

// src/hooks/useChat.ts
export const useChat = () => {
  // Implement custom hook as specified:
  // - State management for conversations and messages
  // - API integration with error handling
  // - Optimistic updates and caching
};
```

**Step 3: Performance Optimization Implementation**
Implement all performance optimizations as specified:

```typescript
// webpack.config.js or vite.config.ts
export default {
  // Implement code splitting as specified
  optimization: {
    splitChunks: {
      chunks: 'all',
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: 'vendors',
          chunks: 'all',
        },
      },
    },
  },
  // Implement all optimization strategies from specifications
};

// src/components/LazyComponents.tsx
const HeavyComponent = lazy(() => import('./HeavyComponent'));

// Implement PWA features as specified
// - Service worker for offline functionality
// - Web app manifest
// - Push notifications (optional)
```

**Step 4: Testing and Accessibility Implementation**
Implement comprehensive testing and accessibility as specified:

```typescript
// src/components/__tests__/ChatInterface.test.tsx
import { render, screen, userEvent } from '@testing-library/react';
import { axe, toHaveNoViolations } from 'jest-axe';

expect.extend(toHaveNoViolations);

describe('ChatInterface', () => {
  it('should be accessible', async () => {
    const { container } = render(<ChatInterface />);
    const results = await axe(container);
    expect(results).toHaveNoViolations();
  });

  // Implement all test cases as specified
});

// Implement accessibility features as specified:
// - ARIA labels and roles
// - Keyboard navigation
// - Screen reader support
// - Focus management
```

**Build and Deployment**:
```bash
# Install dependencies
npm install

# Run development server
npm run dev

# Build for production
npm run build

# Run tests
npm test

# Run accessibility tests
npm run test:a11y

# Deploy to S3 and invalidate CloudFront
aws s3 sync dist/ s3://chatbot-frontend-bucket --delete
aws cloudfront create-invalidation --distribution-id <id> --paths "/*"
```

**Implementation Checklist**:
- [ ] Design system implemented with all tokens and components
- [ ] Component architecture matches specifications exactly
- [ ] Performance optimizations implemented and validated
- [ ] PWA features working (offline, installable)
- [ ] All tests passing with required coverage
- [ ] WCAG 2.1 AA compliance validated
- [ ] Cross-browser compatibility tested
- [ ] Core Web Vitals targets met

Use Agent Hooks to automatically run tests when components are modified and validate design system consistency.
~~~

Expected Outcome

- Complete React application matching all frontend specifications

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences